

PROJECT AND TEAM INFORMATION

Project Title

Online–Offline Banking Transaction Management System (OBTMS)

Student / Team Information

Team Name / Team ID	Tech Titans – DSCPP-III-2026-T143
Shrey Chandola Team Lead Roll No: 2028144	2510010882@geu.ac.in 
Sahil Singh Team Member Roll No: 2028065	2510011026@geu.ac.in 
Shreyank Rautela Team Member Roll No: 2028151	2510012342@geu.ac.in 
Milan Team Member Roll No: 2027914	2510012148@geu.ac.in 

PROPOSAL DESCRIPTION

Motivation

The Online–Offline Banking Transaction Management System (OBTMS) is designed to solve the problem of performing banking transactions during temporary network unavailability. Normal banking systems mostly depend on continuous internet connectivity, which can make transactions difficult when the network is unavailable. OBTMS provides a controlled offline transaction facility while keeping the server as the main authority for account and transaction information.

- . Supports both online and controlled offline transactions.
- . Allows limited transactions during temporary network failure.
- . Uses offline authorization and transaction limits to prevent misuse.
- . Stores offline transactions as pending transactions.
- . Synchronizes pending transactions when the connection is restored.
- . Uses server-side validation and reconciliation to maintain accurate account information.
- . Prevents duplicate transactions using unique transaction IDs.

State of the Art / Current Solution

Most banking applications use online client-server systems in which the user's request is sent to a central server for authentication, validation and transaction processing. These systems provide accurate and secure transactions but generally depend on a stable internet connection. When connectivity is unavailable, users may not be able to perform transactions.

- . Existing banking systems mainly depend on continuous internet connectivity.
- . Transactions are normally verified and processed by a central server.
- . Limited or no transaction facility is available during network failure.
- . OBTMS introduces controlled offline transactions using offline authorization.
- . Offline transactions are stored as pending and processed after reconnection.
- . The server performs validation, synchronization and reconciliation before final settlement.
- . Unique transaction IDs help prevent duplicate transaction processing.

Project Goals and Milestones

General Goal:

The main goal of OOBTMS is to develop a C++-based banking system that supports both online transactions and controlled offline transactions. The project will focus on secure transaction handling, offline authorization, synchronization and proper management of banking data.

- . Develop a client-server banking system using C++.
- . Implement user authentication and account management.
- . Provide secure online money-transfer functionality.
- . Implement controlled offline transactions with predefined limits.
- . Store offline transactions in a pending queue.
- . Synchronize pending transactions when the network is restored.
- . Implement transaction validation, reconciliation and rollback.
- . Prevent duplicate transactions using unique transaction IDs.
- . Apply OOP, DSA, networking and file-handling concepts.

Milestones:

1. Requirement analysis and system design.
2. Design classes and required data structures.
3. Develop client-server communication.
4. Implement authentication and online transactions.
5. Implement offline authorization and pending transactions.
6. Develop synchronization and reconciliation.
7. Testing, debugging and final project demonstration.

Project Approach

The project will be made in C++ using a client-server setup. The client will take input from the user and send requests to the server. The server will keep the account details and transaction records.

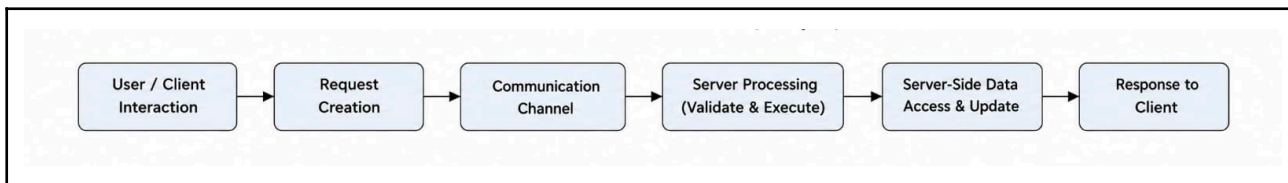
When the server is available, the request will be sent to it and checked before the transaction is carried out. If the connection is weak or gets interrupted, the program will use timeout and retry methods instead of assuming that the transaction was completed.

If the server cannot be reached, a new money transfer will not be approved because the program cannot check the latest account balance. The user will be asked to try again when the connection is available.

Each transaction will have its own ID and status. Requests that need to wait will be kept in a queue. Once the connection is restored, the server will check the pending request again before processing it.

`unordered_map`, `unordered_set`, `queue`, and `vector` will be used for account lookup, checking duplicate transactions, managing pending requests, and storing transaction records. The project will use C++, OOP, DSA, socket-based networking, and file handling.

System Architecture (High Level Diagram)



Project Outcome / Deliverables

The project will deliver a functional Online–Offline Banking Transaction Management System (OOBTMS) that demonstrates how banking transactions can be managed during both normal and temporary network failure. The system will provide controlled offline transactions and safely synchronize them with the server after reconnection.

Expected Deliverables:

- . Working C++ client-server banking application.
- . User authentication and account management.
- . Online money-transfer facility.
- . Controlled offline transaction facility with limits.
- . Offline transaction authorization and pending queue.
- . Persistent storage of accounts and transactions.
- . Synchronization after network recovery.
- . Transaction validation and reconciliation.
- . Settled/Failed status and rollback for unsuccessful transactions.
- . Unique transaction IDs and duplicate transaction detection.
- . Transaction history with basic search and sorting.
- . Project documentation, testing and final demonstration.

Assumptions

The project is developed as an academic banking simulation and is not connected to real banking networks or real money. The system assumes that the server is the authoritative source for account and transaction information, while the client provides controlled services during temporary network failure.

- . Users must be authenticated before performing transactions.
- . The server maintains the authoritative account and transaction data.
- . Offline transactions require a valid offline authorization.
- . Offline transactions have predefined amount and exposure limits.
- . Offline authorization has a fixed expiry period.
- . Pending transactions are stored locally until network connectivity is restored.
- . Final transaction settlement occurs after server-side validation and reconciliation.
- . Deposits and sensitive account changes require an online connection.
- . The system uses simulated money and is intended only for academic purposes.
- . Real banking APIs, payment gateways, advanced security and regulatory compliance are outside the project scope.

References

1. Bjarne Stroustrup – The C++ Programming Language, Addison-Wesley.
2. Robert Lafore – Object-Oriented Programming in C++, Sams Publishing.
3. Mark Allen Weiss – Data Structures and Algorithm Analysis in C++, Pearson.
4. C++ Standard Library Documentation – vector, queue, unordered_map, unordered_set and file handling.
5. Computer Networking Concepts – Client-Server Architecture, TCP/IP and Socket Communication.
6. Transaction Management Concepts – Transaction Processing, Synchronization, Reconciliation and Recovery.
7. File Handling Concepts in C++ – Persistent storage and reading/writing transaction records.